GRAPHCAST–MAMBA: LEARNING NON-MARKOVIAN CORRECTIONS FOR WEATHER FORECASTING

Anonymous authors
Paper under double-blind review

[suozhi: The title is too general. Please name Mamba explicitly and emphasize its ability to capture non-Markovian effects in weather prediction.]

ABSTRACT

Many existing weather forecasting models are Markovian in their chosen state representation, yet unresolved atmospheric processes can make coarse-grained dynamics history dependent. We introduce GraphCast–Mamba, which augments a frozen, pretrained 1° GraphCast model with a correction branch combining graph message passing and temporal memory. On a development set of 32 matched forecast initializations, selected configurations reduce the GraphCast normalized weighted mean-squared error (MSE; GraphCast loss) by approximately 16% over a ten-day rollout and 23% at day 10 (240 h after initialization). These results highlight the potential of learned non-Markovian corrections for improving weather forecasts at longer lead times.

[suozhi: The abstract is too short and should be expanded.]

[suozhi: Please explain terms such as "day ten" at their first occurrence so readers can understand the forecast horizon.]

1 INTRODUCTION

Many numerical and learned forecasters advance a resolved weather state from its current value or a short window of recent values. Examples include the numerical Integrated Forecasting System (IFS) configuration used to produce ERA5 (ECMWF, 2016; Hersbach et al., 2020), and learned Pangu-Weather (Bi et al., 2023), GraphCast (Lam et al., 2023), FuXi (Chen et al., 2023), ECMWF's ERA5-native-grid AIFS (Lang et al., 2024), and WeatherNext 2 (Alet et al., 2025), and WeatherNext 3 (Rasp et al., 2026). This is a practical representation for autoregressive prediction, but this comes with the limitation that the resolved variables are naturally cut-off at some finite resolution scale. State-of-the-art architectures typically operate at spatial resolutions of approximately 1° (~ 111 km), 0.25° (~ 28 km), and 0.1° (~ 11 km) in latitude/longitude, with the latter used by the recent WeatherNext 3 model. Higher resolution typically improves forecast accuracy, but comes at a dramatically increased computational cost and poses an additional challenge for obtaining sufficiently high-resolution training targets, as observational data are sparse and heterogeneous and global reanalysis products are generally available only at substantially coarser resolutions.

On the other hand, the difficulty of resolving all relevant scales is familiar from turbulent-flow simulation. Direct numerical simulation (DNS) discretizes the governing equations finely enough to represent dynamically active scales, and its grid requirements grow rapidly with Reynolds number. For example, a grid-point estimate for a spatially developing turbulent boundary layer gives $N_{\mathrm{grid}} \sim Re_{L_x}^{2.05}$ under locally adapted DNS spacing (Yang & Griffin, 2021). Resolution also affects what a simulation can represent: insufficient small-scale resolution can leave mean dissipation and enstrophy nearly converged while strongly underestimating higher-order gradient statistics and distribution tails (Donzis et al., 2008). Coarser practical models must represent the effects of unresolved motions through an effective closure.

Such a closure need not depend only on the present resolved state. Projecting full dynamics onto a smaller set of variables can yield an instantaneous contribution, a contribution depending on the history of those variables, and a term associated with unobserved initial conditions. The Mori–Zwanzig formalism makes this decomposition explicit; data-driven studies have learned its memory

contribution in reduced dynamical systems (Lin et al., 2021; De Wit et al., 2026). This motivates temporal memory as a possible way to represent effects left out by a finite-window predictor. It does not imply that a learned latent state can be identified with a unique physical memory kernel.

[suozhi: Please define what it means to capture non-Markovian effects and use that definition consistently in the introduction and memory experiments. The evidence supports useful history dependence beyond the baseline's two-frame input. It does not identify a specific physical memory kernel.]

Recent partial differential equation (PDE) experiments provide a concrete example of when memory can help. Buitrago Ruiz and colleagues introduce Memory Neural Operator (MemNO), which inserts a structured state-space sequence (S4) layer within a neural operator and report substantially lower forecast error on coarse or noisy observations. For the Kuramoto–Sivashinsky equation, the advantage of memory over a memoryless operator changes with grid resolution, and the crossover depends on viscosity and the associated high-frequency content (Buitrago Ruiz et al., 2025). Whether an analogous crossover exists for global weather forecasting, and at what grid resolution it might occur, remains an empirical question.

Here, we take a step toward addressing this question by testing whether temporal memory can improve forecasts of some of the frontier models. To test this, we augment a frozen GraphCast model with a trainable Mamba correction branch that combines spatial message passing with Mamba layers. This branch carries a temporal memory state across autoregressive forecast steps and adds a correction to each GraphCast prediction. Learning a residual correction focuses training on what the predictor misses instead of requiring a new model to relearn its dominant dynamics.

In particular, we use the following ansatz for the predictor:

$$(c_t, m_t) = R_\phi(X_t, m_{t-1}), \qquad \widehat{Y}_t = G_\theta(X_t) + c_t, \qquad \theta \text{ fixed}, \tag{1}$$

where G_θ is the pretrained GraphCast backbone, R_ϕ is the trainable Mamba correction branch, and c_t is its forecast correction. The temporal memory state m_t persists between forecast calls and comprises all state-space model (SSM) states and causal-convolution caches in the branch.

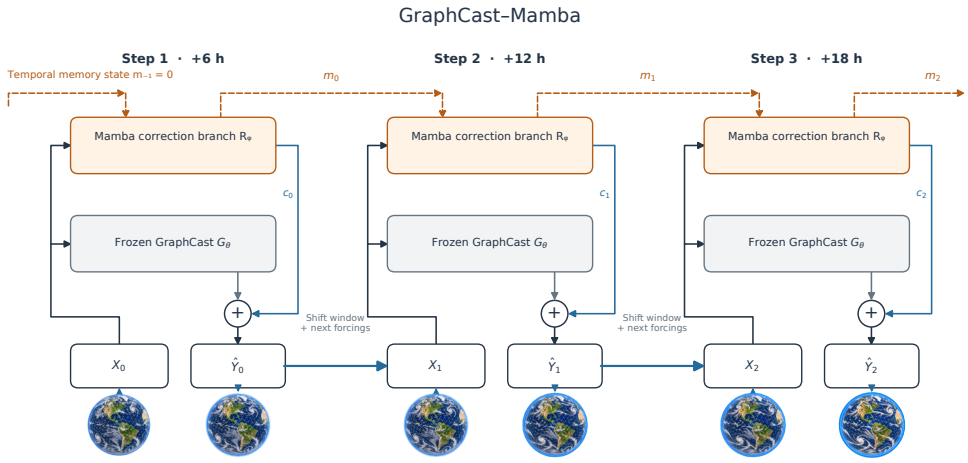
Output corrections to frozen time-series forecasters have been studied in δ-Adapter (Liang et al., 2026), and neural corrections have been applied during the integration of a numerical weather model (Chen et al., 2022). Memory within a forecaster is also established, including recurrent global weather models (Hu et al., 2023) and the neural-operator approach above. These precedents motivate a more specific question: can a separate correction branch improve an already trained state-of-the-art forecaster by carrying temporal memory across its autoregressive forecast steps?

The predictor and correction meet at the output, so this formulation does not require access to the predictor's internal layers. In our weather instantiation, G_θ is a frozen GraphCast model (Lam et al., 2023) and R_ϕ combines graph message passing with Mamba layers (Gu & Dao, 2023). A zero-initialized output head makes the GraphCast–Mamba model start from the GraphCast forecast; only the correction branch is then trained.

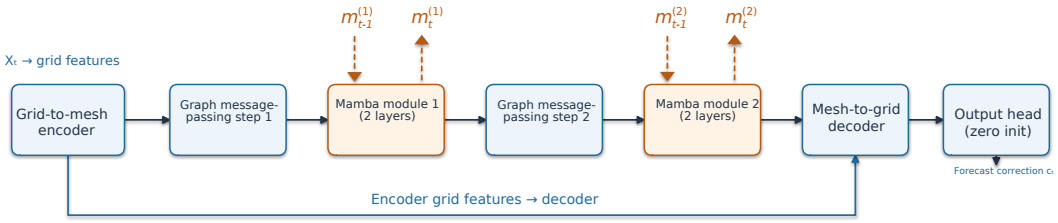
We test this instantiation at 1° resolution on 32 matched forecast initializations from a development set. The selected correction reduces the exact GraphCast loss by 16.34% over ten-day rollouts and 23.68% at day 10. Separate memory-reset controls, measured with an approximate reconstruction of that loss, support a contribution from persistent temporal context beyond the branch's additional trainable capacity. These results establish a case for memory-based correction of a strong pretrained forecaster at the tested resolution. The resolution at which its memory benefit might diminish, and whether the same correction strategy transfers to other forecasters, require separate matched experiments.

[suozhi: Please end the introduction with "Our contributions are threefold." (1) State what new model we introduce. (2) Explain which limitation of previous models the new model addresses and how. (3) Report how the results improve, specifying the baseline, evaluation setting, and metric. Make these three contributions explicit rather than leaving readers to infer them from separate paragraphs.]

[suozhi: Please add a dedicated Related Work section and aim to finish it by approximately the end of page three. You can ask AI to examine all papers cited by a few papers you know best as a starting point for the literature review.]



(a) Autoregressive rollout over three six-hour steps. Each GraphCast–Mamba forecast advances the two-frame input window; dashed orange arrows carry the temporal memory state, initialized here with $m_{-1} = 0$.
[suozhi: Please add a results plot to Figure 1(a) comparing our new model with existing models across forecast horizons. The plot should highlight the new model's improvements at long forecast horizons.]



(b) Trainable Mamba correction branch. Two graph message-passing steps each feed a two-layer Mamba module, followed by decoding and an output head initialized to zero.

Figure 1: GraphCast–Mamba architecture. A frozen GraphCast forecast $G_\theta(X_t)$ is combined with the forecast correction c_t to produce $\hat{Y}_t$. The correction branch has separate parameters and a temporal memory state m_t, comprising the SSM states and causal-convolution caches of its four Mamba layers.

[suozhi: After Related Work, organize the main text into Method and Results, covering approximately pages four through nine.]

[suozhi: Consider using `\vspace` to reduce excess vertical whitespace.]

2 A HYBRID GRAPHCAST–MAMBA ARCHITECTURE

[suozhi: Please prioritize the training and inference procedure of our model. Explain which parameters are frozen, how predictions are fed back, when memory is initialized and carried, and where gradients propagate or are stopped. Move these essential details from the appendix into Method. Shorten the general Mamba derivation or move it to the appendix.]

We study whether temporal memory improves forecasts beyond the finite input history supplied to a pretrained GraphCast model. GraphCast uses two successive atmospheric states as input; its forecast is Markovian with respect to this augmented state. Figure 1a shows how a separate trainable branch corrects each GraphCast forecast and carries memory across successive six-hour steps. The GraphCast–Mamba prediction advances the meteorological input window, while the branch's temporal memory retains information from the preceding trajectory. Figure 1b details how graph message passing mixes information across mesh nodes and Mamba updates memory independently at each node. This introduces history dependence relative to the baseline input representation.

GraphCast The baseline is a pretrained GraphCast checkpoint. This graph neural network operates on a latitude–longitude grid and an auxiliary mesh, with graph $\mathcal{G}(\mathcal{V}^{\mathrm{G}}, \mathcal{V}^{\mathrm{M}}, \mathcal{E}^{\mathrm{G2M}}, \mathcal{E}^{\mathrm{M}}, \mathcal{E}^{\mathrm{M2G}})$. The vertices $\mathcal{V}^{\mathrm{G}}$ represent the meteorological grid and $\mathcal{V}^{\mathrm{M}}$ the mesh. Encoding uses grid-to-mesh edges $\mathcal{E}^{\mathrm{G2M}}$, followed by $N = 16$ graph message-passing steps over $\mathcal{E}^{\mathrm{M}}$ and decoding over mesh-to-grid edges $\mathcal{E}^{\mathrm{M2G}}$. Latent node features have width $W_{\mathrm{GC}} = 512$. The mesh provides a more compact representation for spatial processing than the meteorological grid.

Mamba Mamba is an attention-free sequence model based on selective state-space models (SSMs), designed to model long temporal dependencies with computation that scales linearly with sequence length (Gu & Dao, 2023). A conventional continuous-time SSM maps an input sequence $x(t)$ to an SSM state $h(t)$ according to

$$\frac{dh(t)}{dt} = Ah(t) + Bx(t), \qquad y(t) = Ch(t) + Dx(t). \tag{2}$$

Here, A is a diagonal matrix that determines how quickly memory decays or evolves, B and C determines how new inputs are written into the state and readout from the state respectively, and D provides a direct input-to-output skip connection. For a single SSM channel, each of the matrices have fixed dimension $d_{\text{state}} = 16$ and Δ - is a scalar. If A, B, and C are fixed, the same linear dynamical system is applied at every time step. For a discrete step size Δ, its exact discretization is

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad \bar{A} = \exp(\Delta A), \qquad \bar{B} = A^{-1}\left[\exp(\Delta A) - I\right]B. \tag{3}$$

Mamba makes this SSM *selective*: rather than using fixed input and readout matrices and a fixed step size, it computes B_t, C_t, and Δ_t from the current input. Thus, the model can decide which information to retain, overwrite, or expose at each time step.

In our model, Mamba operates independently on the temporal sequence of each GraphCast mesh node. Let $x_t^{\mathrm{in}} \in \mathbb{R}^{W_{\mathrm{GC}}}$ denote the mesh latent feature at a given node and forecast step. A layer-normalized input is projected into two d_{inner}-dimensional branches,

$$[\tilde{x}_t, z_t] = W_{\mathrm{in}}\,\mathrm{LN}(x_t^{\mathrm{in}}), \qquad \tilde{x}_t, z_t \in \mathbb{R}^{d_{\text{inner}}}. \tag{4}$$

The first branch, $\tilde{x}_t$, supplies the SSM input. The second branch, z_t, is a learned output gate: it does not update the SSM state directly, but controls how much of the SSM output is passed onward.

Before entering the SSM, $\tilde{x}_t$ is processed by a depthwise causal one-dimensional convolution with kernel width four:

$$u_{t,d} = \mathrm{SiLU}\left(b_d^{\mathrm{conv}} + \sum_{k=0}^{d_{\text{conv}}-1} w_{d,k}^{\mathrm{conv}}\,\tilde{x}_{t-k,d}\right), \qquad d = 1, \ldots, d_{\text{inner}}, \tag{5}$$

where $\mathrm{SiLU}(a) = a/(1 + e^{-a})$. "Depthwise" means that each of the d_{inner} channels has its own d_{conv} learned convolution weights $w_{d,k}^{\mathrm{conv}}$ and does not mix with other channels at this stage. "Causal" means that u_t depends only on the current input and the three previous inputs, never on future inputs. During autoregressive inference, the preceding three projected inputs are retained in a convolution cache, so this local temporal context continues across forecast calls.

The selective parameters are generated from u_t by learned linear projections:

$$[r_t, B_t, C_t] = W_x u_t, \qquad r_t \in \mathbb{R}^{d_t}, \quad B_t, C_t \in \mathbb{R}^{d_{\text{state}}}, \tag{6}$$

$$\Delta_t = \mathrm{softplus}(W_\Delta r_t + b_\Delta) \in \mathbb{R}^{d_{\text{inner}}}. \tag{7}$$

Here $d_t = 32$ and $d_{\mathrm{state}} = 16$. The projection matrices, convolution kernels, and biases are trainable parameters optimized using the forecasting loss. In contrast, B_t, C_t, and Δ_t are input-dependent values generated anew at each time step from u_t. The softplus function ensures $\Delta_t > 0$.

The continuous-time state matrix is diagonal and parameterized as

$$A_{d,n} = -\exp(A_{\log,d,n}), \qquad d = 1, \ldots, d_{\text{inner}}, \quad n = 1, \ldots, d_{\text{state}}. \tag{8}$$

Because every element of A is negative, the corresponding factor $\exp(\Delta_{t,d}A_{d,n})$ lies between zero and one, giving stable, decaying memory. The implementation uses the following selective recurrence:

$$h_{t,d,n} = \exp(\Delta_{t,d}A_{d,n})\,h_{t-1,d,n} + \Delta_{t,d}B_{t,n}u_{t,d}. \tag{9}$$

The first term retains a scaled portion of the previous state, while the second writes the current input into memory. A small $\Delta_{t,d}$ gives slow decay and long memory; a large $\Delta_{t,d}$ gives faster decay and a stronger update from the current input.

The state is read out using the selective vector C_t and a learned skip parameter D:

$$s_{t,d} = \sum_{n=1}^{16} C_{t,n} h_{t,d,n} + D_d u_{t,d}. \tag{10}$$

Finally, the gate branch z_t modulates this readout, and the result is projected back to the W_{GC}-dimensional GraphCast latent space:

$$o_t = W_{\text{out}} \left[s_t \odot \text{SiLU}(z_t) \right], \qquad x_t^{\text{out}} = x_t^{\text{in}} + o_t. \tag{11}$$

The principal experiments use $d_{\text{inner}} = 16$, an SSM state dimension of 16 per channel, a causal convolution width of 4, and a rank-32 projection for Δ_t; we also examine $d_{\text{inner}} = 32$ and 64. The SSM state h_t and the three-token convolution cache persist separately for each batch element, mesh node, and Mamba layer across autoregressive forecast steps. The equations above describe one shared group of input-dependent B_t, C_t coefficients. The grouped variant partitions the inner channels into groups with separate B_t, C_t coefficients; the temporal-memory ablation compares one and four such groups.

An additive correction with temporal memory. Let X_t denote the two-frame meteorological input and the associated forcings, and let m_t collect the SSM states and convolution caches of all Mamba layers. As shown in Figure 1a, the forecast combines the frozen GraphCast backbone G_θ with a trainable correction R_ϕ:

$$(c_t, m_t) = R_\phi(X_t, m_{t-1}), \qquad \hat{Y}_t = G_\theta(X_t) + c_t, \qquad \theta \text{ fixed}. \tag{12}$$

The correction branch in Figure 1b has its own grid-to-mesh encoder, mesh processor, and mesh-to-grid decoder. It receives the same meteorological inputs as the baseline and uses separate parameters throughout. Its mesh processor uses two graph message-passing steps, each followed by a two-layer Mamba module, giving four recurrent layers in total. Encoder grid features are also passed directly to the decoder. Suppressing this grid-feature connection in the notation, the mesh-processing path is

$$L_t^{(0)} = \text{Encode}_\phi(X_t), \tag{13}$$

$$\widetilde{L}_t^{(k)} = \text{GNN}_\phi^{(k)}(L_t^{(k-1)}), \tag{14}$$

$$(L_t^{(k)}, m_t^{(k)}) = \text{MambaModule}_\phi^{(k)}(\widetilde{L}_t^{(k)}, m_{t-1}^{(k)}), \qquad k = 1, 2, \tag{15}$$

$$c_t = \text{Decode}_\phi(L_t^{(2)}). \tag{16}$$

Each Mamba module contains the residual Mamba updates described above; $m_t^{(k)}$ comprises the SSM states and convolution caches of its two layers, while h_t denotes an individual SSM state. The output head is initialized to zero, so the initial GraphCast–Mamba forecast equals the GraphCast forecast. During autoregressive forecasting, the oldest input frame is dropped, the GraphCast–Mamba prediction $\hat{Y}_t$ is appended, and the known forcings are updated to form X_{t+1}. The temporal memory state is passed separately to the next correction call. The cold-start rollout illustrated in Figure 1a begins with $m_{-1} = 0$ and retains the updated temporal memory state thereafter.

3 Numerical Results

We focus on 1° GraphCast models and ask two questions: does a learned correction improve forecast skill at longer lead times, and does temporal memory contribute beyond the additional trainable capacity? We first report forecast skill, then isolate the effect of state carry, and finally examine sensitivity to training choices.

3.1 Evaluation protocol and forecast loss

[suozhi: Please add a compact experiment overview table listing the model, forecast starts, verification grid, and metric for each study. Clearly separate the exact GraphCast loss evaluations, the approximate

reconstructed loss, and the historical comparison across resolutions. Readers should not assume that every figure uses the same model and evaluation protocol. Consistently distinguish loss reduction from RMSE reduction.]

The evaluations on matched forecast initializations use 32 forecast initializations from 2022 and 40 six-hour forecast steps, corresponding to ten days. Forecasts start with a zero temporal memory state and use the full GraphCast–Mamba prediction as feedback, without a history warm start. The pretrained GraphCast baseline remains frozen during correction training. The principal correction branch has spatial width 512, two graph message-passing steps, and Mamba inner dimension 16.

Our primary metric is the exact GraphCast loss, including its difference-standard-deviation normalization, latitude weights, pressure-level weights, and variable weights. If $\ell_{i,k}^{\mathrm{model}}$ and $\ell_{i,k}^{\mathrm{GC}}$ are the losses for forecast initialization i and lead step k, the rollout loss reduction is

$$\mathcal{I}_{1:40} = 100 \left(1 - \frac{\sum_{i=1}^{32} \sum_{k=1}^{40} \ell_{i,k}^{\mathrm{model}}}{\sum_{i=1}^{32} \sum_{k=1}^{40} \ell_{i,k}^{\mathrm{GC}}} \right). \tag{17}$$

Day-10 reduction uses only $k = 40$. We aggregate losses before taking the ratio; these percentages are neither RMSE reductions nor arithmetic averages of per-lead percentages. Each evaluated model uses its saved matched baseline. The older reset study reconstructs an approximate loss from channel root-mean-square errors (RMSEs) and is explicitly identified as such below.

3.2 Improved skill at longer forecast lead times

Figure 2 shows the $d_{\mathrm{inner}} = 16$ correction trained with a $3\times$ Mamba learning rate and SWA. It reduces exact aggregate ten-day loss by 16.34% and day-10 loss by 23.68%. The per-lead reduction grows from 3.45% at day 4 to 16.25% at day 7 and 23.68% at day 10. Day-1 loss reduction is -0.13%, indicating a small degradation; short-lead skill is largely preserved rather than uniformly improved. At day 10, 2 m temperature RMSE is 2.29 K, a 10.98% reduction relative to the paired baseline.

Figure 3 shows that the gains at longer forecast lead times extend across all 11 predicted variable groups. The correction was trained on 15 years of ERA5 data (2007–2021), with $K = 20$ and $d_{\mathrm{inner}} = 128$; the 32 cold-start forecast initializations used for evaluation in 2022 differ from those in Figure 2. At every displayed lead from 120 to 240 h, the GraphCast–Mamba model has lower latitude-weighted RMSE than frozen GraphCast in every variable group, with reductions increasing with lead in each group. At day 10, the reductions range from 10.6% to 15.5%: 10.6% for 2 m temperature, 13.4% for mean sea-level pressure, 12.6% for 6 h accumulated precipitation, and 15.5% for 10 m zonal wind. The upper-air variables also improve, including geopotential, temperature, specific humidity, horizontal wind, and pressure vertical velocity. These results demonstrate broad gains in forecast skill at longer lead times across the surface and upper-air state, measured directly by variable-group RMSE.

We further assess forecast skill for the meteorological quantities in Figure 4, considered in the original GraphCast study (Lam et al., 2023). These panels compare forecasts on a common 1.5° verification grid and again show more favorable error growth with forecast lead time for our corrected 1° model than for its uncorrected baseline. Note that the model output resolution and verification resolution are different quantities. Remarkably, in this comparison, the corrected 1° model achieves lower errors at longer forecast lead times even than the GraphCast (0.25°) reference, despite its coarser spatial resolution and the substantial computational investment in training the reference model: approximately four weeks on 32 TPU v4 devices (Lam et al., 2023).

Appendix D illustrates individual forecast trajectories at Paris and New York City, including both improvements and larger local errors.

3.3 Evidence for useful temporal memory

[suozhi: Please give the memory controls a more prominent role in the argument. A small table should show whether carry/carry, carry/reset, and reset/reset retain memory during training and forecasting, and which control is trained separately. Explain how the controls distinguish additional model capacity from persistent memory. Since resetting clears both the SSM state and convolution cache, the current study does not isolate the contribution of long-term SSM memory.]

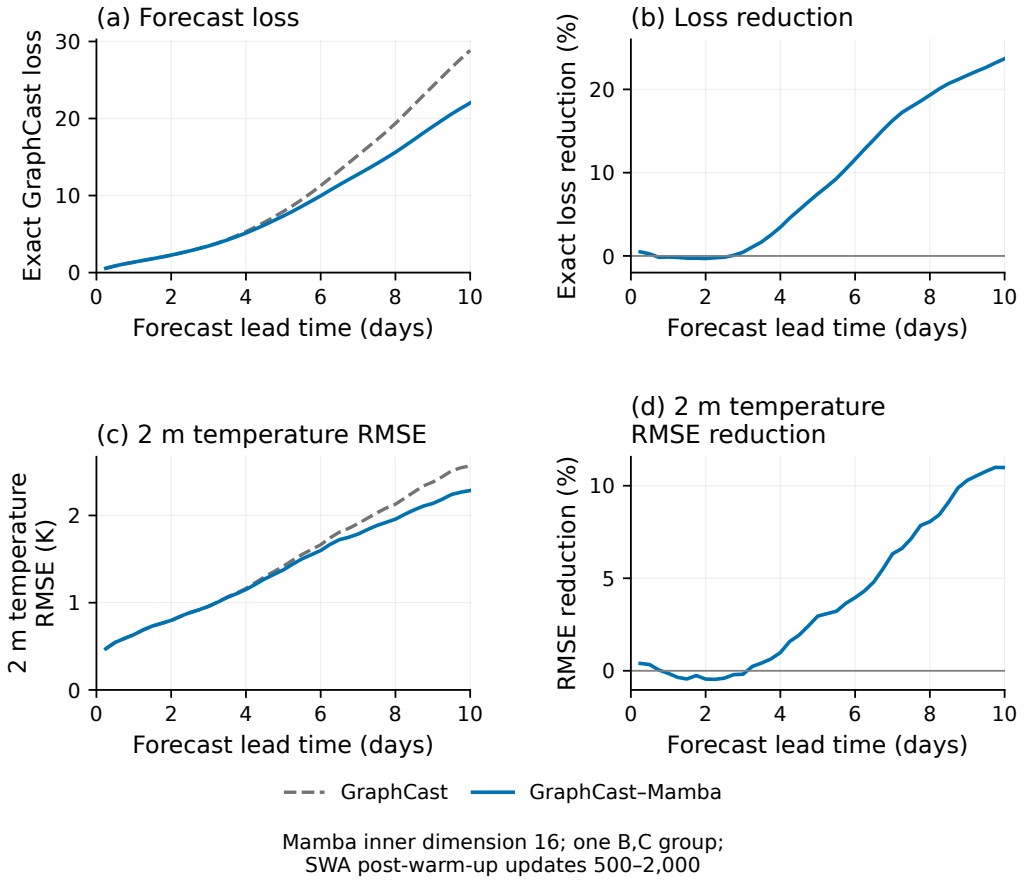


Figure 2: Forecast skill of the $d_{\text{inner}} = 16$ correction trained with a threefold Mamba learning rate (LR) and stochastic weight averaging (SWA) at 1° on 32 matched forecast initializations. Top: exact GraphCast loss and per-lead loss reduction. Bottom: cosine-latitude-weighted 2 m temperature RMSE (K) and RMSE reduction on the same forecast initializations. Reductions are relative to its paired GraphCast baseline. Training and model-selection details are given in Appendix B.

A correction branch can improve forecasts through additional spatial capacity as well as temporal memory. To distinguish these contributions, we compare three policies: state carry during both training and forecasting; state carry during training followed by resetting at every forecast step; and a separately trained model with resetting at every training and forecast step. Resetting zeros both the SSM state and the convolution cache. The carry-trained and reset-trained runs have the same architecture, data settings, optimizer, and training seed; their training/forecasting memory policy differs.

Figure 5 isolates the training/forecasting memory policy for a fixed $d_{\text{inner}} = 16$ model. Carry/carry gives the strongest improvement at longer forecast lead times: day-10 approximate loss reduction is 22.52%, compared with 9.24% for carry/reset and 12.91% for reset/reset. Retaining memory therefore adds 13.28 percentage points relative to resetting the trained state during forecasting and 9.62 percentage points relative to training and forecasting without persistent temporal memory. Additional group settings and averaging windows are reported in Appendix A.

The forecast-time reset demonstrates dependence on the learned state, but also introduces a training–forecasting mismatch. The separately trained reset control is therefore essential: it retains the trainable correction architecture while removing persistent temporal context. Its smaller benefit at longer forecast lead times supports a contribution from memory beyond additional correction capacity. This is evidence for useful history dependence relative to the baseline's two-frame input representation.

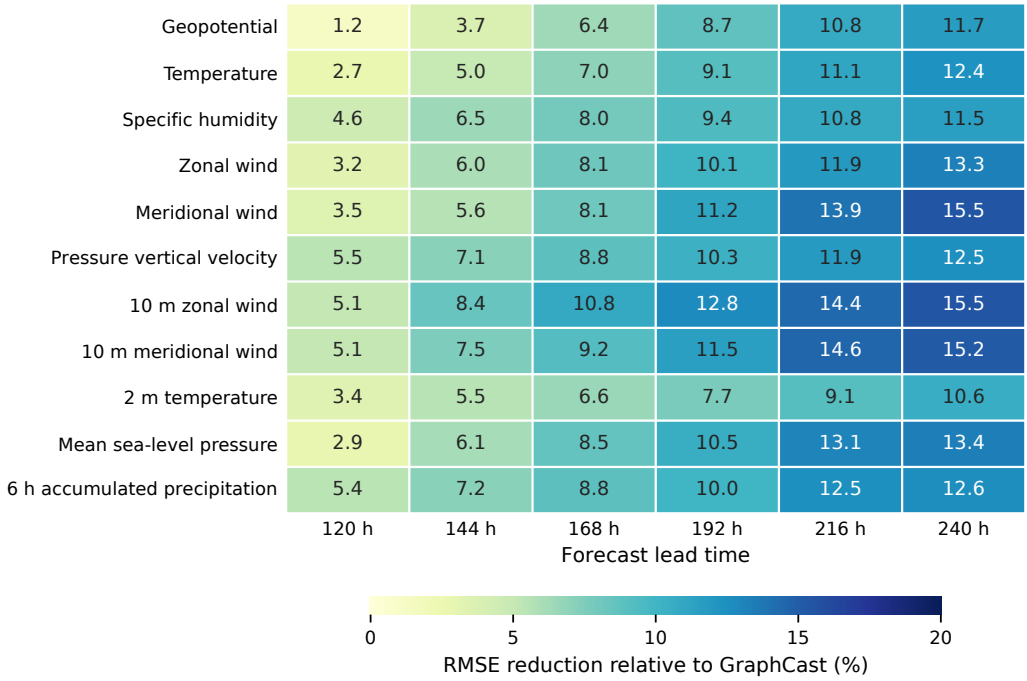


Figure 3: Variable-group RMSE reduction relative to frozen GraphCast at forecast leads of 120–240 h. Each cell reports $100\,(1 - \mathrm{RMSE}_{\mathrm{ours}}/\mathrm{RMSE}_{\mathrm{GraphCast}})$, with positive values indicating lower error for the GraphCast–Mamba forecast. RMSE is computed in each variable's physical units, using cosine-latitude weighting and averaging squared errors across forecast initializations before taking the square root. For upper-air variables, squared errors are also averaged equally across the 13 pressure levels. This correction was trained separately on 15 years of ERA5 data (2007–2021), with a training tail of $K = 20$ feedback-conditioned steps, Mamba inner dimension 128, spatial width 512, and two graph message-passing steps; SWA averages checkpoints from updates 6k–11k. Evaluation comprises 32 forecast initializations in 2022, drawn from 30 January–11 February, 26 May–7 June, and 20 September–2 October. The temporal memory state is initialized to zero, and each GraphCast–Mamba prediction is fed back in full for the next step. These initialization dates differ from the matched set used in Figure 2.

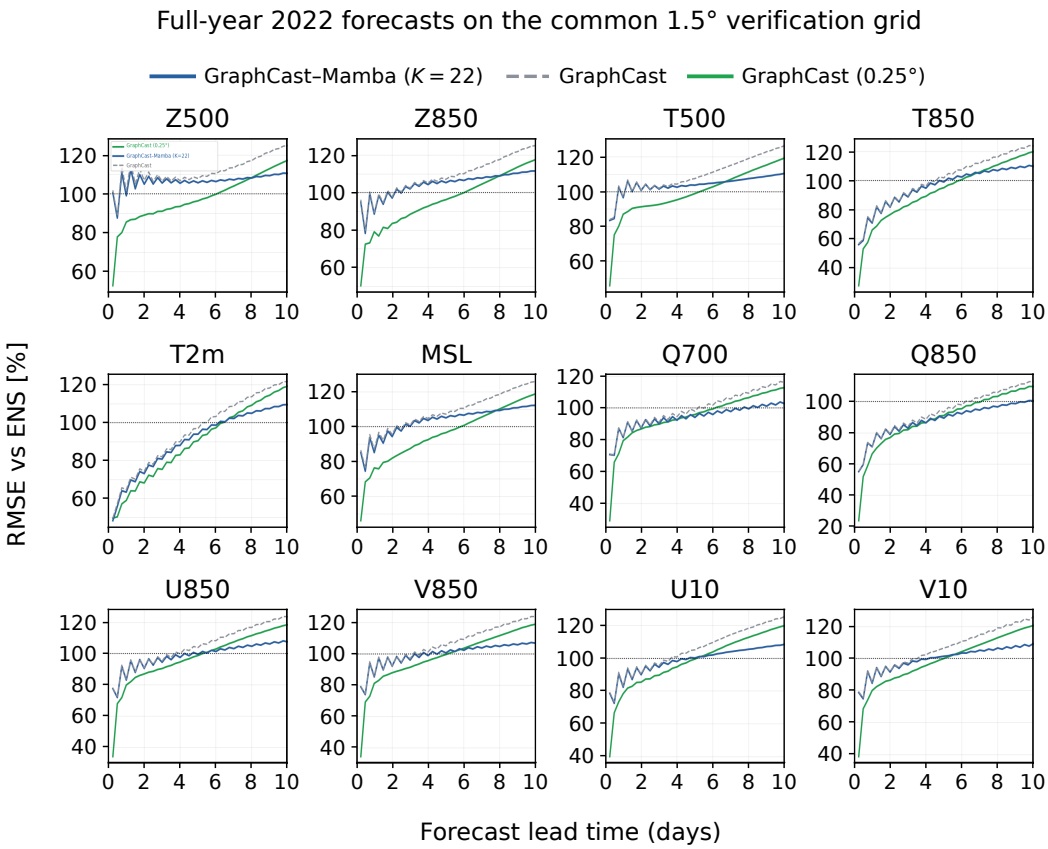


Figure 4: Complete common-grid comparison. The quantities span geopotential, temperature, mean sea-level pressure, specific humidity, and wind. The panel labels use Z for geopotential, T for temperature, Q for specific humidity, U for zonal wind, and V for meridional wind; a three-digit suffix gives pressure in hPa. T2m denotes 2 m temperature, U10 and V10 denote the 10 m wind components, and MSL denotes mean sea-level pressure. Blue is the archived $K = 22$ GraphCast–Mamba configuration, dashed gray its GraphCast baseline, and green GraphCast ($0.25°$). Forecasts are verified on a common $1.5°$ grid, with RMSE normalized to the ECMWF ensemble (ENS) reference (100% denotes equal RMSE). This historical comparison is separate from the evaluations on matched forecast initializations in Figure 2.

The experiment does not identify a unique physical memory kernel or isolate the SSM contribution from that of the convolution cache.

3.4 Sensitivity to inner dimension and Mamba learning rate

Table 1 compares inner dimensions 16 and 32 under two training methods: equal spatial and Mamba learning rates and a $3\times$ Mamba learning rate with the spatial learning rate unchanged. All four configurations use the same training schedule and SWA window. Under equal spatial and Mamba learning rates, the dimension-32 correction achieves larger aggregate and day-10 loss reductions than the dimension-16 correction. With a $3\times$ Mamba learning rate, the ordering reverses: the dimension-16 correction shown in Figure 2 achieves the largest reductions in the table. The benefit of increasing inner dimension therefore depends on the Mamba learning rate in these runs. Each run starts from its own trained warm-up parameters, so these are comparisons of complete training runs. Appendix B gives the training details and the additional $0.3\times$ Mamba learning-rate results.

Scope of the evidence. The exact-loss studies use one training seed and a 32-initialization development set used in model selection. We therefore report descriptive improvements without significance

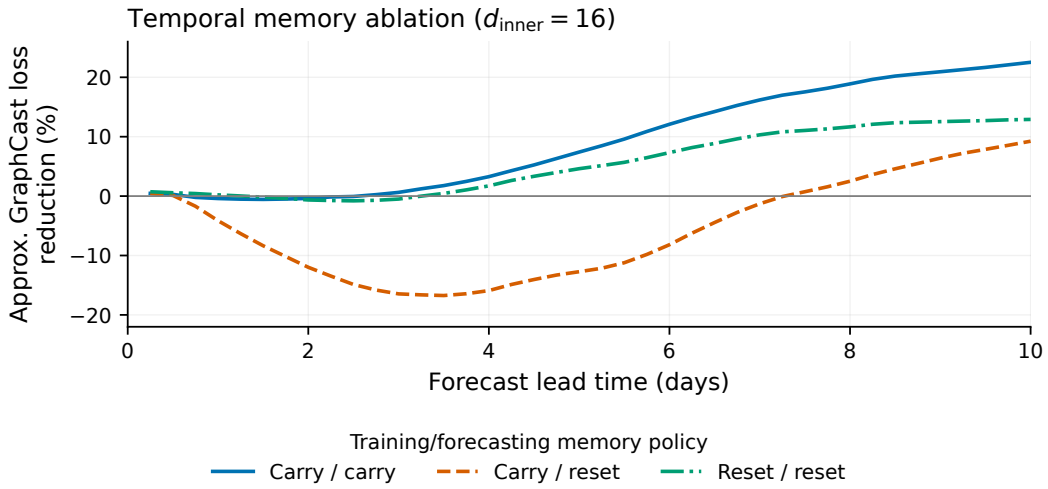


Figure 5: Temporal memory improves forecasts at longer lead times. All three policies use the same architecture with $d_{\text{inner}} = 16$, one B, C group, and SWA over updates 2k–8k. Carry/carry retains state during training and forecasting; carry/reset clears state during forecasting; reset/reset clears it during both. Carry/carry achieves the largest day-10 loss reduction. Results use 32 matched cold-start forecast initializations and 40 six-hour forecast steps; positive values indicate lower loss than the paired GraphCast baseline. Loss is reconstructed approximately from saved channel RMSEs.

Table 1: Inner dimension and Mamba learning-rate comparison at 1° on the matched 32-initialization development evaluation. Entries are exact GraphCast loss reductions (%); bold marks the largest value in each metric column. All runs use spatial width 512, one B, C group, batch size four, the same training schedule, and SWA over post-warm-up updates 500, 1,000, 1,500, and 2,000. The spatial learning rate is unchanged; $1\times$ denotes equal spatial and Mamba learning rates.

d_{inner}	Mamba LR multiplier	Ten-day aggregate	Day 10
16	$1\times$ (equal)	15.67	22.44
32	$1\times$ (equal)	16.29	23.32
16	$3\times$	**16.34**	**23.68**
32	$3\times$	15.73	22.75

claims. The memory controls support the contribution of persistent state in the tested configuration; they do not directly establish the same attribution for every model. Evaluation on forecast dates reserved from selection, with matched uncertainty estimates and exact-loss reset controls, would strengthen the quantitative conclusions.

AI USE STATEMENT

We used a large language model for minor editorial assistance, including correcting grammar and typo and refining formatting. It did not contribute to the development of research ideas, the analysis, or the claims.

ETHICS STATEMENT

All datasets, models in this work are publicly available. The usage of these datasets, models, code strictly follows the terms and conditions set by the providers.

REPRODUCIBILITY STATEMENT

To ensure reproducibility of our results, we provide detailed documentation of our experimental setup and methodology in the main paper and appendix. The code for data, evaluation, and training will be open-sourced upon publication.

[suozhi: We need to include an AI use statement before submitting the paper.]

REFERENCES

Ferran Alet, Ilan Price, Andrew El-Kadi, Dominic Masters, Stratis Markou, Tom R. Andersson, Jacklynn Stott, Remi Lam, Matthew Willson, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Skillful joint probabilistic weather forecasting from marginals, 2025. URL https://arxiv.org/abs/2506.10772. WeatherNext 2 technical report.

Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Accurate medium-range global weather forecasting with 3d neural networks. *Nature*, 619:533–538, 2023. doi: 10.1038/s41586-023-06185-3. URL https://doi.org/10.1038/s41586-023-06185-3.

Ricardo Buitrago Ruiz, Tanya Marwah, Albert Gu, and Andrej Risteski. On the benefits of memory for modeling time-dependent PDEs. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/89379d5fc6eb34ff98488202fb52b9d0-Abstract-Conference.html.

Lei Chen, Xiaohui Zhong, Feng Zhang, Yuan Cheng, Yinghui Xu, Yuan Qi, and Hao Li. FuXi: a cascade machine learning forecasting system for 15-day global weather forecast. *npj Climate and Atmospheric Science*, 6:190, 2023. doi: 10.1038/s41612-023-00512-1. URL https://doi.org/10.1038/s41612-023-00512-1.

Tse-Chun Chen, Stephen G. Penny, Jeffrey S. Whitaker, Sergey Frolov, Robert Pincus, and Stefan Tulich. Correcting systematic and state-dependent errors in the NOAA FV3-GFS using neural networks. *Journal of Advances in Modeling Earth Systems*, 14:e2022MS003309, 2022. doi: 10.1029/2022MS003309. URL https://doi.org/10.1029/2022MS003309.

Xander M De Wit, Alessandro Gabbana, Michael Woodward, Yen Ting Lin, Federico Toschi, and Daniel Livescu. Data-driven mori–zwanzig modeling of lagrangian particle dynamics in turbulent flows. *Proceedings of the National Academy of Sciences*, 123(13):e2525390123, 2026.

D. A. Donzis, P. K. Yeung, and K. R. Sreenivasan. Dissipation and enstrophy in isotropic turbulence: Resolution effects and scaling in direct numerical simulations. *Physics of Fluids*, 20(4):045108, 04 2008. ISSN 1070-6631. doi: 10.1063/1.2907227. URL https://doi.org/10.1063/1.2907227.

ECMWF. *IFS Documentation CY41R2, Part III: Dynamics and Numerical Procedures*. European Centre for Medium-Range Weather Forecasts, 2016. doi: 10.21957/83wouv80.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Hans Hersbach, Bill Bell, Paul Berrisford, et al. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049, 2020. doi: 10.1002/qj.3803. URL https://doi.org/10.1002/qj.3803.

Yuan Hu, Lei Chen, Zhibin Wang, and Hao Li. SwinVRNN: A data-driven ensemble forecasting model via learned distribution perturbation. *Journal of Advances in Modeling Earth Systems*, 15: e2022MS003211, 2023. doi: 10.1029/2022MS003211. URL https://doi.org/10.1029/2022MS003211.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization, 2019. URL https://arxiv.org/abs/1803.05407.

Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382 (6677):1416–1421, 2023. doi: 10.1126/science.adi2336. URL https://doi.org/10.1126/science.adi2336.

Simon Lang, Mihai Alexe, Matthew Chantry, Jesper Dramsch, Florian Pinault, Baudouin Raoult, Mariana C. A. Clare, Christian Lessig, Michael Maier-Gerber, Linus Magnusson, Zied Ben Bouallègue, Ana Prieto Nemesio, Peter D. Dueben, Andrew Brown, Florian Pappenberger, and Florence Rabier. AIFS—ECMWF's data-driven forecasting system, 2024. URL https://arxiv.org/abs/2406.01465.

Daojun Liang, Qi Li, Yinglong Wang, Jing Chen, Hu Zhang, Xiaoxiao Cui, Qizheng Wang, and Shuo Li. The forecast after the forecast: A post-processing shift in time series. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/6f7d90b1198fec96defd80b5ebd5bc81-Abstract-Conference.html.

Yen Ting Lin, Yifeng Tian, Daniel Livescu, and Marian Anghel. Data-driven learning for the mori–zwanzig formalism: A generalization of the koopman learning framework. *SIAM Journal on Applied Dynamical Systems*, 20(4):2558–2601, 2021. doi: 10.1137/21M1401759. URL https://doi.org/10.1137/21M1401759.

Stephan Rasp, Boris Babenko, Dominic Masters, Andrew El-Kadi, Samier Merchant, Guy Shalev, Ilan Price, Fred Zyda, Remi Lam, Sasha Shysheya, Matthew Willson, Stratis Markou, Shreya Agrawal, Suhani Vora, Mohammed Alewi Hassen, Sunny Mak, Tom R. Andersson, Megan Bela, Akib Uddin, Nofar Peled Levi, Ben Gaiarin, Ferran Alet, Aaron Bell, Peter Battaglia, and Alvaro Sanchez-Gonzalez. WeatherNext 3: Increasing resolution and performance of global weather models with raw observations, 2026. URL https://arxiv.org/abs/2609.03582.

Alasdair Tran, Alexander Mathews, Lexing Xie, and Cheng Soon Ong. Factorized Fourier neural operators. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=tmIiMPl4IPa.

Xiang IA Yang and Kevin P Griffin. Grid-point and time-step requirements for direct numerical simulation and large-eddy simulation. *Physics of Fluids*, 33(1), 2021.

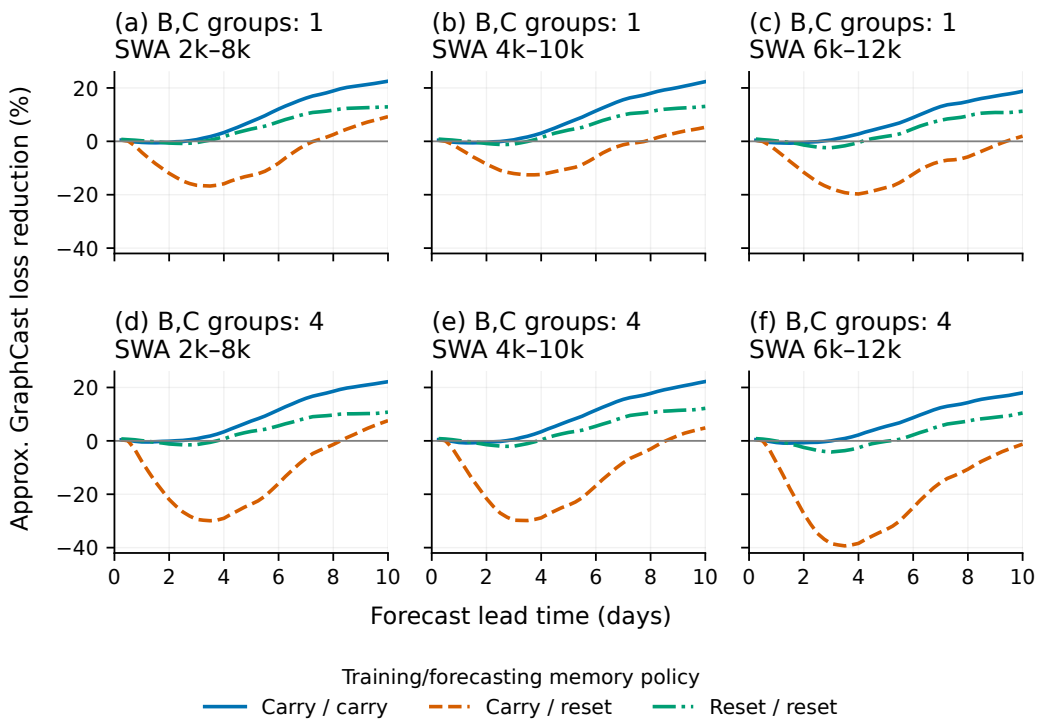


Figure 6: Complete temporal-memory ablation. Column titles specify the SWA windows, and rows specify one or four B, C groups. All panels use approximate GraphCast loss reconstructed from saved channel RMSEs.

A Temporal-memory ablation across averaging windows

The full temporal-memory ablation fixes $d_{\text{inner}} = 16$ and varies the number of B, C groups (one or four) and the SWA window (2k–8k, 4k–10k, or 6k–12k updates). Carry/carry achieves the largest day-10 reduction in each panel of Figure 6. The SWA windows overlap and are derived from the same training trajectories; they are sensitivity checks, not independent seed repetitions. The approximate metric and the distinct training/forecasting memory policies are the same as in Section 3.3. Each curve uses its saved baseline (maximum baseline channel-RMSE difference across the full study: 0.56%); exact pole weighting cannot be recovered from these channel summaries.

B Training choices and checkpoint averaging

Training methods and model selection. Table 1 reports four SWA configurations from the Mamba learning-rate study: equal spatial and Mamba learning rates and a $3\times$ Mamba learning rate, each with inner dimensions 16 and 32. The equal-rate configuration applies the same learning rate to both parameter groups; the $3\times$ configuration increases only the Mamba rate. The pretrained GraphCast parameters remain frozen in all cases. Figure 2 uses the dimension-16, $3\times$ Mamba learning-rate SWA configuration, which has the largest aggregate and day-10 exact-loss reductions among the completed SWA configurations in the reported optimizer and learning-rate studies. This selection uses the matched 32-initialization development evaluation and does not establish optimality on an independent test set.

Optimizer study. The optimizer study varies peak learning rate (10^{-4} or 5×10^{-5}), Adam moments, temporal initialization scheme (legacy or Mamba1), and the paired inner-dimension/group settings (16/1 or 64/4). The latter changes two factors simultaneously and does not isolate inner dimension. Runs use 10,000 updates, seed 22, a 200-update warm-up, cosine decay to one tenth of the peak

learning rate, and weight decay 10^{-4}. The correction is trained with closed-loop feedback, 24-step backpropagation windows, a 20-step autoregressive tail, and loss across all 24 steps. The correction contribution to the feedback is stop-gradient, while temporal memory state gradients propagate within the backpropagation window.

Each run screens periodic checkpoints and three SWA windows on eight forecast initializations. The selected checkpoint and SWA are then evaluated on the matched 32 forecast initializations. Selection uses these development evaluations, so the study does not represent an untouched test-set comparison.

Day-10 reductions for the 22 completed selected SWA configurations range from 13.47% to 23.21%. The configuration at the upper end uses Mamba1 initialization, inner dimension 16, one B, C group, learning rate 10^{-4}, and AdamW moments $(0.9, 0.98)$; its aggregate loss reduction is 16.28%.

Mamba learning rate and inner dimension. The Mamba learning-rate study keeps spatial width 512 and one B, C group while varying inner dimension and the Mamba learning-rate multiplier. Each arm uses its own 500-update warm-up followed by 2,000 post-warm-up updates at batch size four. The base peak learning rate is 10^{-4} with AdamW moments $(0.9, 0.98)$. Table 2 extends the four configurations in Table 1 with the $0.3\times$ Mamba learning rate at each inner dimension. All six use SWA over post-warm-up updates 500, 1,000, 1,500, and 2,000. The warm-up parameter values differ across arms despite the common training seed, so the comparisons describe complete training runs rather than isolated learning-rate interventions.

Under equal spatial and Mamba learning rates, the dimension-32 SWA correction exceeds the dimension-16 correction's aggregate loss reduction by 0.62 percentage points. Under the $3\times$ Mamba learning rate, the dimension-32 correction instead has a smaller aggregate reduction by 0.61 percentage points. The $0.3\times$ Mamba learning-rate SWA results also favor dimension 32, while each dimension attains its largest reductions at a different multiplier: $3\times$ for dimension 16 and $1\times$ for dimension 32. These observations do not establish a monotonic benefit from increasing inner dimension or Mamba learning rate.

Table 2: Exact GraphCast loss reduction (%) for the learning-rate study's SWA models. Spatial learning rate is unchanged across these six arms; the multiplier applies to Mamba parameters. All evaluations contain the same 32 forecast initializations.

d_{inner}	Mamba LR multiplier	Ten-day aggregate	Day 10
16	$0.3\times$	14.70	21.98
16	$1\times$	15.67	22.44
16	$3\times$	**16.34**	**23.68**
32	$0.3\times$	15.82	22.96
32	$1\times$	16.29	23.32
32	$3\times$	15.73	22.75

Stochastic weight averaging. SWA averages model parameter values from saved checkpoints to reduce dependence on a single training iterate (Izmailov et al., 2019). Every configuration in Tables 1 and 2 averages four checkpoints at post-warm-up updates 500, 1,000, 1,500, and 2,000. The dimension-16 Mamba1-initialized optimizer-study configuration described above averages seven checkpoints, saved every 1k updates from 2k through 8k. These are parameter averages, not ensembles of independent forecast predictions. Evaluation initializes the temporal memory state to zero, including for SWA models.

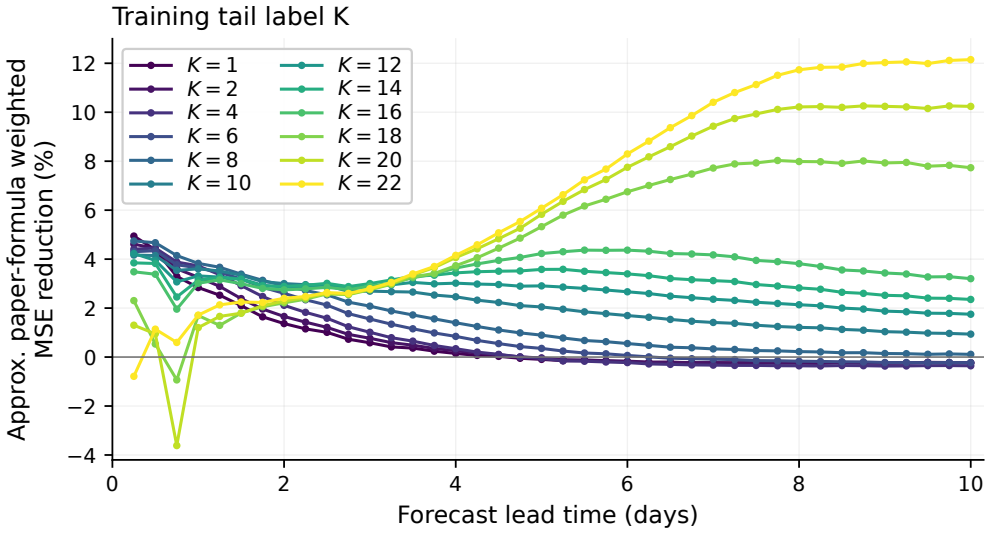


Figure 7: Dependence of forecast skill on the training tail label K in the archived v22 sweep. Each curve corresponds to a separately trained correction and shows the approximate paper-formula weighted MSE reduction relative to its saved GraphCast baseline over 40 six-hour forecast steps. This diagnostic is reconstructed from archived RMSE summaries and differs from the exact GraphCast loss used in Figure 2. The largest change in skill at longer forecast lead times occurs between $K = 16$ and $K = 18$.

C Training tail length and the breadth of forecast gains

This appendix examines the archived v22 training-tail sweep, whose saved evaluations each contain 32 forecast initializations and 40 six-hour forecast steps. These historical models are distinct from the configurations in the exact-loss studies in the main text. We report the archived weighted-MSE diagnostic for the training-tail comparison and latitude-weighted RMSE for the variable comparison.

C.1 Dependence on training tail length

Previous neural-operator studies, including MemNO and Factorized Fourier Neural Operators (F-FNO), have demonstrated effective forecasting using teacher forcing, where predictions are conditioned on ground-truth states during training (Buitrago Ruiz et al., 2025; Tran et al., 2023). MemNO retains a history of these states: its next-step supervision therefore includes temporal context. Here we examine how forecast skill depends on the training tail label K. We retain the historical labeling convention: $K = 1$ denotes fully teacher-forced training, whereas $K > 1$ denotes K feedback-conditioned steps in each 24-step training window. The latter follow a teacher-forced prefix, so the longest chain includes $K + 1$ predictions, beginning with the final prediction from an ERA5 input.

Figure 7 reveals a strong dependence on training tail length. Shorter training tails primarily improve early forecasts, with gains diminishing as the forecast advances. Longer training tails produce substantially larger improvements at longer forecast lead times, with a pronounced crossover between $K = 16$ and $K = 18$ in the present sweep. This behavior suggests that sufficiently long training tails are important for realizing the correction model's benefit at longer forecast lead times. We interpret this as an empirical crossover for this configuration; establishing a universal critical training tail length would require further experiments.

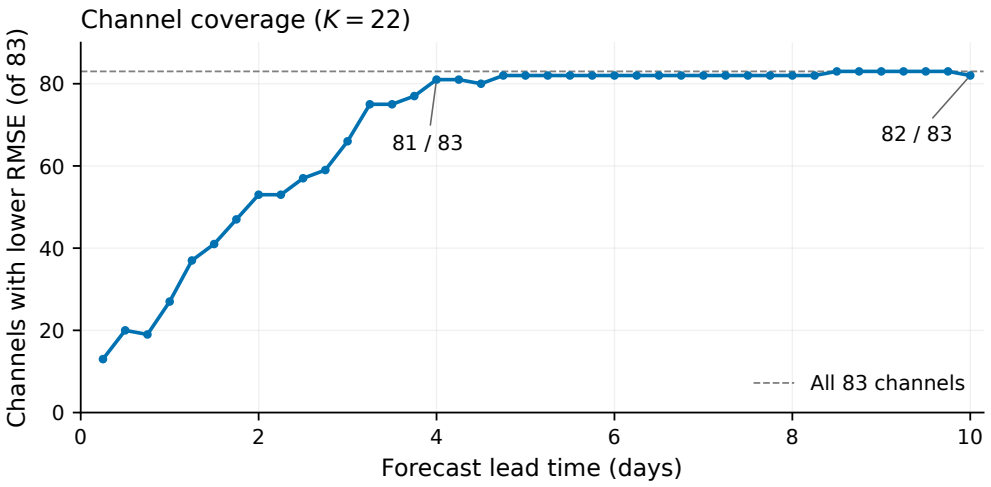


Figure 8: Breadth of forecast improvements for the archived $K = 22$ correction. At each lead, a channel is counted when its latitude-weighted RMSE is lower than that of its paired GraphCast baseline. The 83 channels comprise five surface variables and six upper-air variables at 13 pressure levels. The count reaches 81 at day 4, remains at 82–83 from 114 h onward, and is 82 at day 10. This counts the signs of the RMSE differences and does not establish statistical significance for individual channels.

C.2 Improvements across forecast variables

For the $K = 22$ model, the gains extend across the predicted atmospheric state. Figure 8 shows that the number of channels with lower latitude-weighted RMSE than GraphCast increases with forecast lead time, reaching 81 of 83 channels at day 4 and remaining at 82–83 channels from approximately day 5 onward. All 83 channels improve at the evaluated leads from 204 to 234 h; 82 improve at 240 h, with 50 hPa temperature the remaining exception.

At day 10, all 11 reported variable groups have lower RMSE, with reductions ranging from 4.55% to 9.66%. The improvements span temperature, geopotential, mean sea-level pressure, specific humidity, precipitation, and wind, including pressure vertical velocity. The distinction between variable groups and individual pressure-level channels matters: improvement in every group can coexist with a small number of unimproved channels. Together, these diagnostics show that the gains at longer forecast lead times extend broadly across the forecast variables.

D FORECAST EXAMPLES AT CITY LOCATIONS

Figure 9 gives a compact illustration of local forecast behavior for temperature, wind, and precipitation. Over 120–240 h, the correction reduces mean absolute error in four of the six displayed city–variable pairs. Errors increase for Paris temperature and New York precipitation, illustrating that aggregate skill gains can coexist with deterioration in individual forecasts.

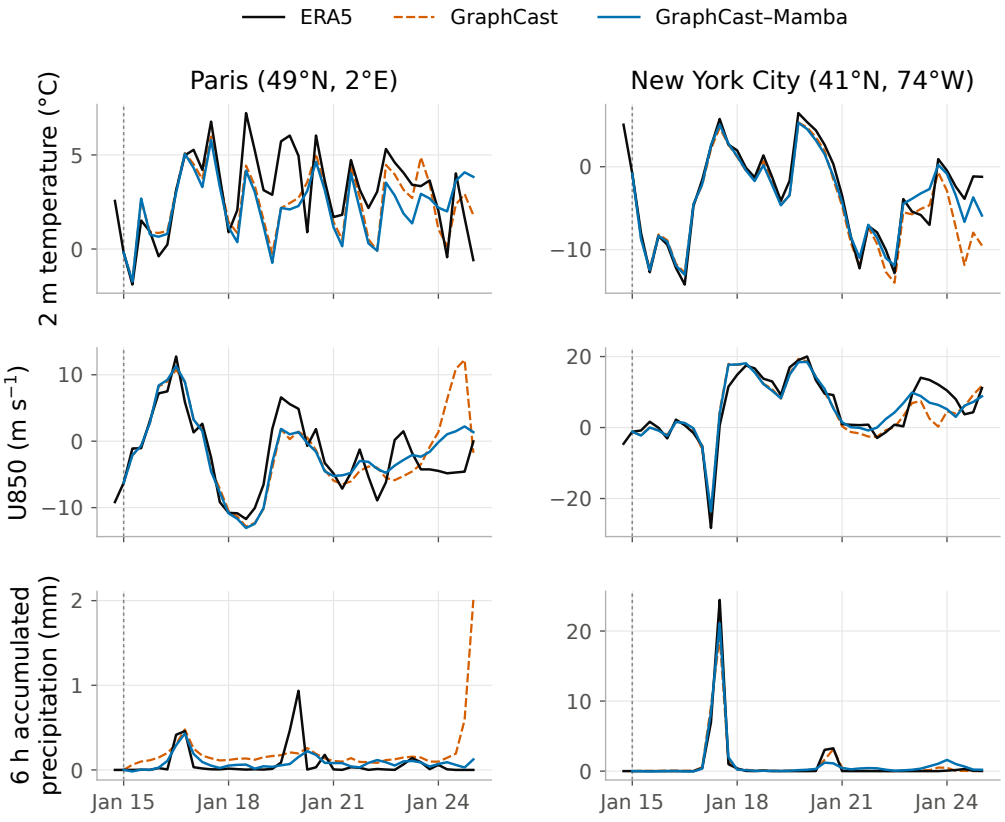


Figure 9: Ten-day forecasts from 00 UTC on 15 January 2022 at the 1° grid points nearest Paris (49°N, 2°E; left) and New York City (41°N, 74°W; right). Rows show 2 m temperature, 850 hPa zonal wind (U850), and 6 h accumulated precipitation in physical units. ERA5 is black, GraphCast is dashed orange, and the GraphCast–Mamba forecast is blue. The correction is the separately trained $K = 20$ model in Figure 3, fitted to 15 years of ERA5 data. Both forecasts use ERA5 states at -6 and 0 h, then feed back their own predictions; the GraphCast–Mamba model begins with a zero temporal memory state. The dotted line marks initialization. Precipitation is shown as predicted and may take small negative values.